Лекция 6. Ошибки аутентификации и JWT





БАЗОВЫЕ ПОНЯТИЯ

Идентификация, аутентификация и авторизация

Веб-сервер не помнит предыдущий HTTP-запрос: протокол HTTP сам по себе не хранит состояние. После успешного входа приложение должно связать следующие запросы с тем же пользователем.

БАЗОВЫЕ ПОНЯТИЯ

Три ключевых понятия

Идентификация

Пользователь **сообщает**, кем он является — например, вводит логин `student`. Это заявление о личности, которое ещё не подтверждено.

Аутентификация

Приложение **проверяет доказательство** личности: пароль, код, ключ или токен. Только здесь происходит реальная верификация.

Авторизация

Приложение **решает**, разрешено ли этому пользователю действие: просмотр профиля, панели администратора или задания.

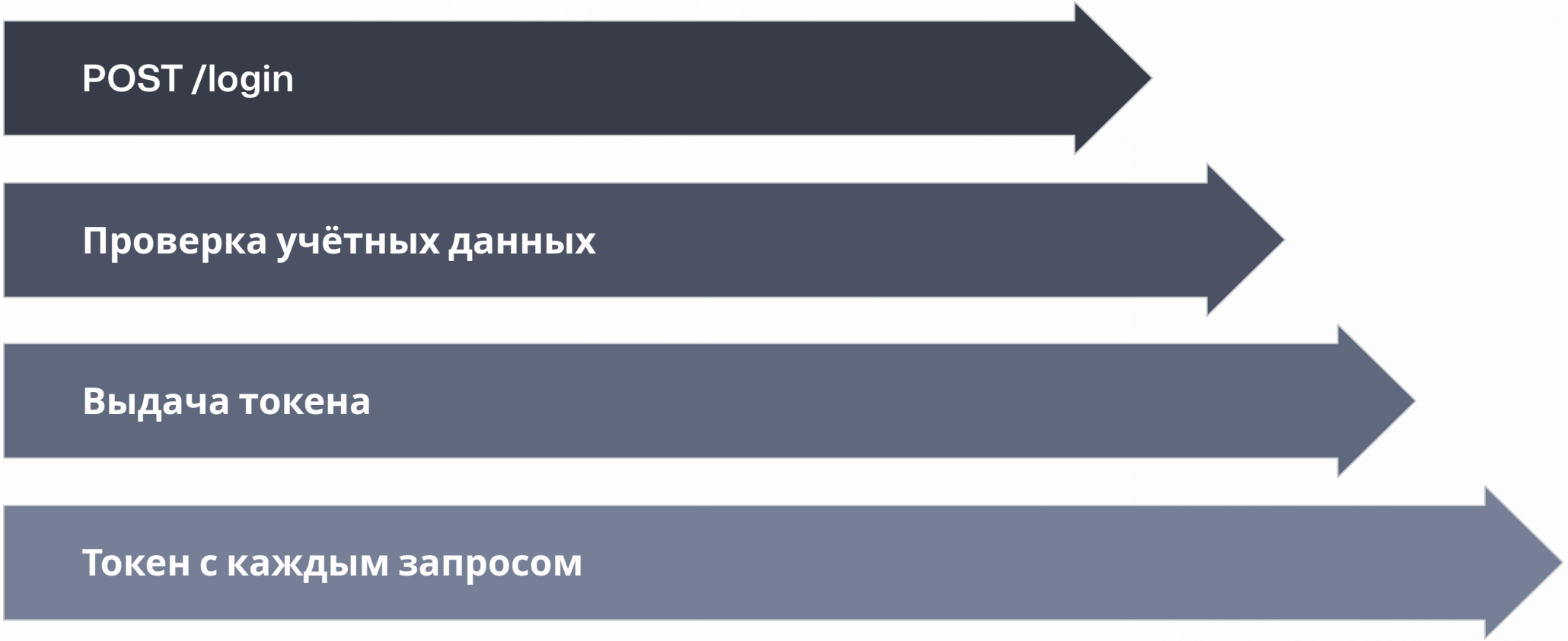
Сессия

Состояние входа, которое **связывает** браузер и сервер между несколькими HTTP-запросами.

Ошибка аутентификации обычно связана не с криптографией как таковой, а с неверным доверием к данным от клиента: изменяемой cookie, неподписанному токену или слабому секрету.

БАЗОВЫЕ ПОНЯТИЯ

Типовая последовательность входа



На каждом этапе могут возникать уязвимости — от слабой проверки пароля до отсутствия валидации токена на стороне сервера.

МЕТОДЫ АУТЕНТИФИКАЦИИ

Методы аутентификации в веб-приложениях

Веб-приложения используют несколько подходов для подтверждения личности пользователя. Понимание каждого метода необходимо для поиска уязвимостей.

01

Логин и пароль

Классический способ через HTTPS с хранением сессии

02

Серверная сессия и cookie

Случайный идентификатор, хранящийся на сервере

03

Токены доступа (JWT)

Самодостаточный токен с подписью в заголовке Authorization

04

OAuth 2.0 / OpenID Connect

Делегирование входа внешнему поставщику идентификации

05

Многофакторная аутентификация

TOTP, WebAuthn или аппаратный ключ в дополнение к паролю

COOKIE И СЕССИИ

Как работают cookie и серверные сессии

Сервер отвечает

```
Set-Cookie: session_id=a4f5...;
  Path=/;
  HttpOnly
```

Сервер создаёт случайный идентификатор, хранит его вместе с данными сессии у себя и передаёт клиенту.

Браузер отправляет

```
Cookie: session_id=a4f5...
```

При каждом последующем запросе браузер автоматически добавляет cookie. Cookie — небольшая пара «имя–значение». Чаще всего в ней лежит только **ключ к серверной сессии**, а не пароль.

COOKIE И СЕССИИ

Атрибуты безопасности cookie

Domain / Path

Определяют, на какие адреса браузер отправляет cookie. Ограничение области действия снижает риск утечки.

Expires / Max-Age

Задают срок хранения. Cookie без срока действия — сессионная и удаляется при закрытии браузера.

Secure

Ограничивает передачу только по HTTPS. Без этого атрибута cookie может быть перехвачена по незащищённому каналу.

HttpOnly

Скрывает cookie от JavaScript в браузере. Критически важно для защиты от кражи через XSS.

SameSite

Влияет на отправку cookie в межсайтовых запросах. Значения `Strict` и `Lax` снижают риск CSRF-атак.

АТАКИ НА СЕССИИ

Сценарий: подмена клиентской cookie

Если приложение хранит роль пользователя прямо в cookie без подписи, атакующий может изменить её вручную.

```
Cookie: user=student; role=user
```

1 Перехватить запрос

Войти с разрешённой тестовой учётной записью и перехватить запрос к защищённой странице через Burp Suite.

2 Найти роль в cookie

Найти значение `role=user` в заголовке Cookie. Убедиться, что значение передаётся открытым текстом.

3 Заменить и повторить

В Burp Repeater заменить на `role=admin`, отправить запрос и сравнить статус, тело ответа и доступные разделы.

Причина успеха: приложение приняло авторизационное решение по данным, которые полностью контролирует браузер.

АТАКИ НА СЕССИИ

Предсказуемый ID сессии и Session Fixation

Предсказуемый идентификатор

Сервер может создавать сессии вида `session-1001`, `session-1002`. Цепочка анализа:

1. Получить cookie до входа и записать значение
2. Войти и проверить, изменился ли идентификатор
3. Создать несколько сессий и сравнить формат
4. Подставить другой известный ID в Burp Repeater

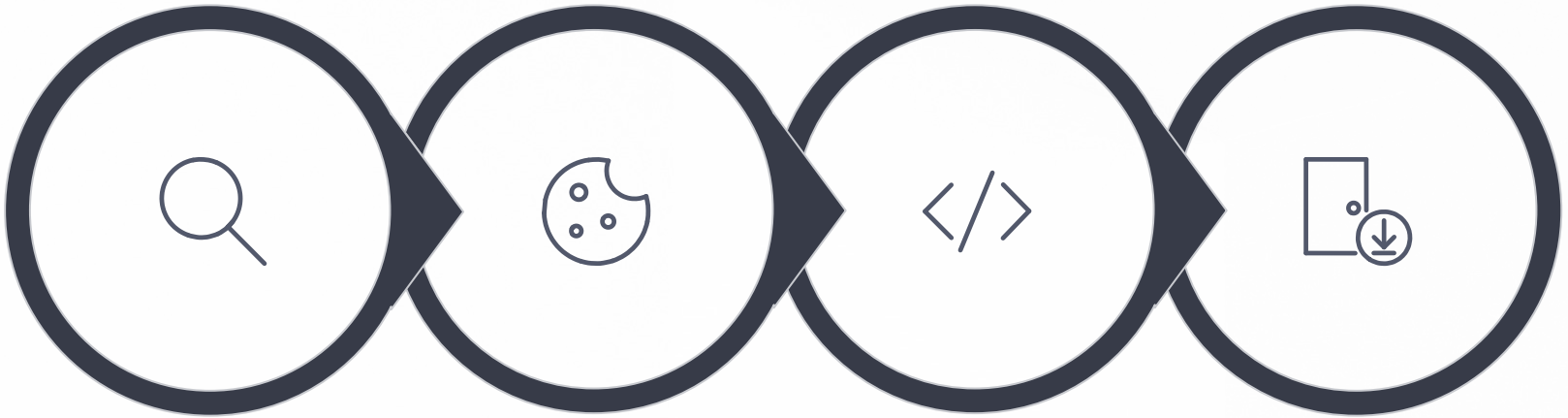
Session Fixation

Атакующий заранее получает **известный ID сессии**, заставляет пользователя пройти вход с ним, а затем использует тот же ID для получения доступа.

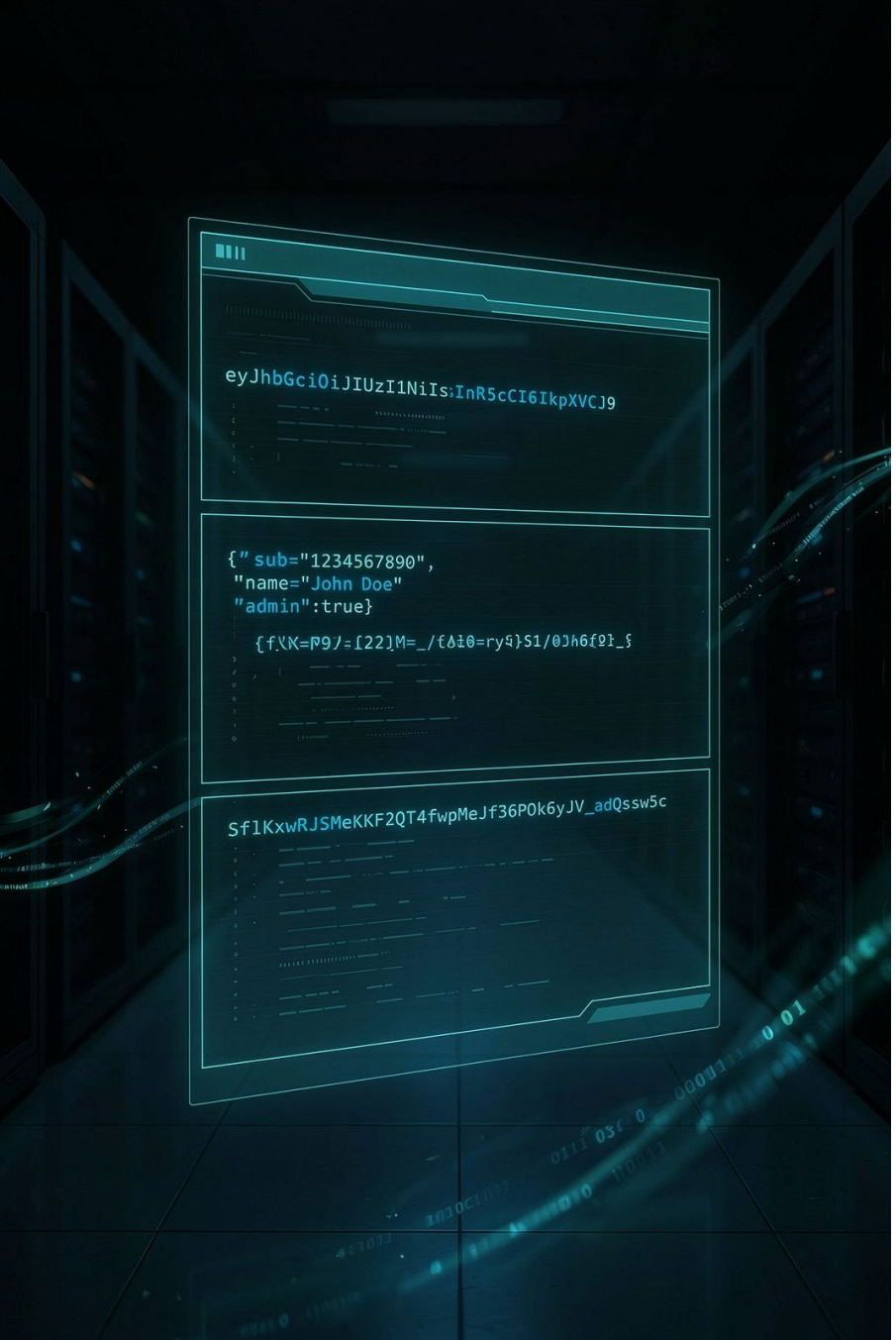
Для анализа достаточно моделировать сценарий двумя тестовыми браузерными профилями или Burp Repeater.

АТАКИ НА СЕССИИ

Кража cookie через XSS



Если приложение выводит введённый текст как HTML и auth-cookie не имеет атрибута `HttpOnly`, JavaScript на странице может прочитать `document.cookie`. Это демонстрирует разницу между XSS (точка выполнения кода) и захватом сессии (последствие).



JWT

JWT: структура и формат

JWT (JSON Web Token) — строка из трёх частей, разделённых точками: `header.payload.signature`. Первые две части закодированы Base64URL — **не зашифрованы**. Их можно декодировать без секрета.

JWT

Анатомия JWT-токена

Header

```
{"alg":"HS256",
 "typ":"JWT"}
```

Payload

```
{"sub":"student",
 "role":"user",
 "exp":1800000000}
```

Распространённые claims в payload

Claim	Значение
`sub`	Субъект токена: ID или логин пользователя
`role`, `scope`	Права, добавленные приложением
`iat`	Время выдачи токена
`exp`	Срок действия токена
`iss`	Издатель токена
`aud`	Аудитория — кому предназначен токен

JWT

Подпись JWT: HS256 vs RS256

HS256 (симметричный)

Сервер и проверяющая сторона используют **общий секрет**. Если секрет слабый или утечёт — токен можно подделать. Вся безопасность зависит от длины и случайности ключа.

RS256 (асимметричный)

Токен подписывается **приватным RSA-ключом**, а проверяется публичным. Компрометация публичного ключа не позволяет подделать подпись, только верифицировать.

Подпись защищает содержимое от незаметной подмены — но **только если** сервер действительно проверяет подпись, ожидаемый алгоритм и важные claims.

АТАКИ НА JWT

Шаг 1: Найти и разобрать токен

→ Выполнить вход в тестовой среде

Войти с разрешённой учётной записью и перехватить трафик через Burp Suite.

→ Найти JWT

Искать в cookie, заголовке `Authorization: Bearer` или теле ответа API.

→ Декодировать токен

Скопировать строку в `jwt.io`, CyberChef или локальный `jwt-tool`.

→ Зафиксировать данные

Записать `alg` из header, пользователя и роль из payload, значения `exp`, `iss`, `aud`.

→ Сохранить исходный запрос

Добавить в Burp Repeater — он нужен для сравнения всех последующих попыток.

АТАКИ НА JWT

Четыре сценария атак на JWT



Нет проверки подписи

Сервер декодирует payload, но не проверяет signature. Изменение `role` принимается без ошибки.



Принятие alg: none

Сервер принимает алгоритм `none` из header — подпись полностью отсутствует после второй точки.



Слабый секрет HS256

Секрет короткий или из словаря — подбирается локально через Hashcat без обращения к серверу.



Неверная проверка claims

Подпись верна, но сервер игнорирует `exp`, `aud` или принимает токен для другого маршрута.

АТАКИ НА JWT

Сценарий: alg:none и слабый секрет

Атака alg: none

1. Взять исходный JWT с `{"alg":"HS256"}`
2. Изменить header на `{"alg":"none","typ":"JWT"}`
3. В payload изменить роль на `admin`
4. Собрать токен: `base64url(header).base64url(payload).` — после второй точки подпись отсутствует
5. Проверить, изменился ли доступ к защищённому ресурсу

Атака слабый секрет HS256

1. Сохранить JWT в файл `token.txt`
2. Убедиться, что алгоритм симметричный (HS256)
3. Запустить Hashcat локально:

```
hashcat -m 16500 token.txt \
 /path/to/wordlist.txt
```

После нахождения секрета подписать токен с изменённым claim `role` и подтвердить изменение доступа.

АТАКИ НА JWT

Неверная проверка claims и путаница алгоритмов

Неверная проверка claims

Подпись может быть правильной, но приложение всё равно ошибается в логике. Проверить: принимает ли сервис **истёкший** токен? Игнорирует ли `aud`? Выдаёт ли доступ только по наличию поля `role` без проверки значения? Отправить старый токен и токен для другого маршрута API — зафиксировать, на каком этапе сервер должен был отклонить запрос, но не сделал этого.

Путаница алгоритмов

В старых или неверно настроенных реализациях сервер может принимать и `RS256`, и `HS256`, хотя должен принимать только один ожидаемый тип. Один и тот же ключ нельзя использовать в разных схемах подписи. Достаточно выявить небезопасную конфигурацию и объяснить риск — это продвинутая тема.

ИНСТРУМЕНТЫ

Инструменты для исследования

DevTools браузера

Cookie, Local Storage, Network, параметры URL

Burp Suite Community

Перехват, изменение и повтор HTTP-запросов в Repeater

OWASP ZAP

Альтернативный прокси и анализатор HTTP

jwt.io

Визуальное декодирование header и payload JWT

CyberChef

Base64URL, JSON и преобразования токенов

jwt-tool

Локальный анализ структуры JWT и вариантов проверки

Hashcat

Проверка JWT HS256 на слабый секрет по словарю

curl / jq

Воспроизведение запросов и удобное чтение JSON из API

ИНСТРУМЕНТЫ

Примеры запросов curl для тестирования

Получить заголовки ответа

```
curl -i https://example.test/
```

Отправить cookie

```
curl -i \
  -H 'Cookie: session_id=<значение>' \
  https://example.test/profile
```

Отправить JWT

```
curl -i \
  -H 'Authorization: Bearer ' \
  https://example.test/api/profile
```

Когда что использовать

- **-i** — показывает заголовки ответа вместе с телом
- **-H** — добавляет произвольный заголовок к запросу
- Флаг **Cookie** — для тестирования сессионных сценариев
- Заголовок **Authorization: Bearer** — стандартный способ передачи JWT в API
- Комбинируйте с **jq** для красивого вывода JSON: `curl ... | jq .`

Итоги лекции

HTTP не хранит состояние

Идентификация, аутентификация и авторизация — три разных процесса. Сессия связывает их между запросами.

Cookie — точка атаки

Атрибуты HttpOnly, Secure и SameSite критически важны. Роль или логин в cookie без подписи — уязвимость.

JWT читается без секрета

Base64URL — не шифрование. Подпись защищает только при корректной проверке алгоритма и всех claims.

Четыре вектора атак на JWT

Нет проверки подписи, alg:none, слабый секрет HS256, неверная проверка claims — каждый требует отдельного анализа.